

Desafío 1

Implementación de SweetCrush mediante manipulación de bits en C++

Este desafío pretende aplicar conceptos de operadores a nivel de bits (bitwise) y gestión de memoria dinámica, bajo restricciones estrictas de programación estructurada; con los objetivos establecidos en la guía del Desafío 1.

Esta parte corresponde a diseñar en el papel una solución que permita administrar un tablero dinámico, con eliminaciones individuales de fichas, detectar combinaciones, modificar su estructura y gestionar diferentes situaciones en pantalla; con fichas independientes de manera inequívoca dentro del sistema.

Descripción y planeación de la solución:

Entendemos que el reto es hacer un juego de tipo SweetCrush, más allá del juego, lo más importante es como se guarda el tablero en memoria (una tira de bits en memoria dinámica); teniendo en cuenta que: cada ficha ocupa 3 bits, hay que evitar usar struct, class, STL ni string, y todo debe hacerse con operadores de bits. Solo debe usarse: `<istream>`, `<stdlib.h>` (rand) y `<time>` (semilla).

La mayor dificultad es mover y manipular bits que no están alineados con los bytes.

- Para codificar las fichas, tenemos en cuenta que cada ficha se representa con un código binario de 3 bits, de tal forma que 000 es vacío, 001, 010, 011, 100, 101 y 110 son las fichas desde 1 hasta 6 y 111 queda libre. La utilidad del 0 (cero) es la representación de vacío, sirve como relleno, es la comparación más sencilla. `rand() % 6 + 1` debe generar las fichas nuevas, y la semilla

se inicializa con $\text{rand}(\text{time}(0))$.

El código 7 se puede usar como marca especial, pero no es obligatorio.

- De fila-columna al bit (tachón real)

$\text{indice} = \text{fila} * \text{columnas} + \text{columna}$ es la fórmula para ubicar una casilla.

Cada casilla ocupa 3 bits, por lo tanto el bit inicial es 3p.

Para saber en que byte cae usamos: $\text{byte} = b/8$, $\text{desp} = b\%8$
por ejemplo bit inicial = $17 * 3 = 51$ localizado en el byte 8. se
Al escribir "byte 8" se tiene que:

$$\text{byte} = 51/8 = 6$$

$$\text{desp} = 51\%8 = 3$$

lo que quiere decir que el bite 51 está en el byte 6 y desplazamiento 3; correspondiente al primer tachón real.

- Para el mapa de bytes, algunas fichas quedan partidas entre dos bytes

bits:	7	6	5	4	3	2	1	0
byte 0:	2	2	1	1	1	0	0	0
byte 1:	5	4	4	4	3	3	3	2
byte 2:	7	7	7	6	6	6	5	5
byte 3:	10	10	9	9	9	8	8	8
byte 4:	•	•	•	•	11	11	11	10

relleno con - que va arriba ala izquierda.

Tabla completa

Casilla	Fila-columna	bit	byte	desplas	situación
0	0-0	0	0	0	Cabe
1	0-1	3	0	3	Cabe
2	0-2	6	0	6	Partida 0-1
3	0-3	9	1	1	Cabe
4	1-0	12	1	4	Cabe
5	1-1	15	1	7	Partida 1-2
6	1-2	18	2	2	Cabe
7	1-3	21	2	5	Cabe
8	2-0	24	3	0	Cabe
9	2-1	27	3	3	Cabe
10	2-2	30	3	6	Partida 3-4
11	2-3	33	4	1	Cabe

Los desplazamientos son 0, 3, 6, 1, 4, 7, 2, 5 y vuelve a empezar cada 8 casillas = 3 bytes exactos (24 bites)

2 de cada 8 quedan partidas equivalente al 25% del tablero, sin importar el tamaño

- Para la ficha partida entre dos bytes, cada vez que esto ocurre, se debe leer y escribir en ambos.

Es decir, que al aplicar movimientos o corrimientos, los bits que desaparecen o "se pierden", son los que deben ir al siguiente byte. Aunque parece un error puede ser correcto.

- Memoria reservada:

se aplica la fórmula $\text{bytes} = \frac{3.F.C + 7}{8}$

El +7 es necesario porque la división entera trunca. De no ser así, falta un bit, asegurando que siempre haya espacio suficiente, aunque sobren algunos bits de relleno.

- Eliminación y Menú

Se puede eliminar: - una ficha desde el menú, - la combinación de tres o más automáticamente o - una fila o columna completa como opción del menú

Pues el menú permite eliminar fichas, filas o columnas, y también agregar filas o columnas.

Después de cada jugada se prefiere mostrar el tablero en binario, en fichas y el estado del juego (dimensiones, eliminaciones, cascadas, puntaje)

Al eliminar una ficha entran nuevas, pero al eliminar una fila o una columna, el juego se encoge (el tablero) sin agregar fichas.

- La cruz o taction real surge apartir de la idea equivocada de marcar las fichas al eliminar con el código 111 directamente en el tablero. Porque al hacerlo se pierden combinaciones cruzadas. Es posible que se corrija usando un arreglo de marcas aparte, con un bit por casilla. Se marcan todas las posiciones y se eliminan de una sola pasada.